

EGCI 213
Group Project 2 – Tour Simulation

The project can be done in a group of ≤ 5 students. Each group must do the project by themselves:

- **Everyone involved in cheating, either as a source or copier, will get ZERO point.**
- If late submitting group copies code from a graded group, the graded group will still be penalized.
- **In case of AI usage, you must write your own prompts and perform your own acquisition with the AI.** Due to the dynamic nature of generative AI, it is very unlikely that any 2 groups will get identical generated code even when using identical prompts.
 - Therefore, submitting identical code will be counted as cheating.
 - Don't use generated content obtained by other groups as your own.
 - Don't share generated content you get from the AI with other groups.
 - If any suspicious arises, I may ask both groups to show their chat history with the AI. Failure to do so will result in cheating penalty.
- And remember, you must be able to orally explain every line of the code you submit, regardless of its origin.

1. This project uses only 1 input file [config.txt](#). Lines beginning with "#" = comments; don't remove them.

```
# First column in every parameter line = label
#
# Label followed by: days of simulation
days,          6
#
# Limousine services: inbound and outbound are in separate lines
# Label followed by: number of services, service capacity
inbound_num_max,    3, 100
outbound_num_max,   2, 200
#
# Label followed by: number of places
place_num,         3
#
# TourThreads
# Label followed by: number of threads, min daily arrival, max daily arrival
tour_num_min_max,   3, 50, 100
#
# LocalGuideThreads
# Label followed by: number of threads
guide_num,         4
```

Don't hard code these values. I may change some of them to check whether your calculation is correct.

- There are always 6 [parameter lines](#) with columns as stated above, but numbers may be changed.
- There won't be any input error (e.g. invalid input, negative number, wrong format, missing columns) in this file. But the program must still handle the case of missing file. Don't let it crash.

2. Implement **class Limousine** representing an individual limousine service. There will be a few CityLimo (i.e. inbound) and a few AirportLimo (i.e. outbound) services. The number of services and service capacity (i.e. max seats) are read from config file. You may keep CityLimos and AirportLimos in separate ArrayLists.

3. Implement **class Place** representing an individual place.

4. Implement **class TourThread** representing an individual tour operator as thread. Thread activities are done in loop. Each iteration of a loop = 1 day. In each day:

- 4.1 Wait until one thread (can be main, TourThread, or GuideThread) prints day number, and remaining seats on each CityLimo and AirportLimo. The remaining seats on each limo is reset to max seats at the beginning of each day.
- 4.2 **On even day**, process outbound customers:
 - 4.2.1 Report the number of outbound customers and wait until all threads complete this step.
 - 4.2.2 Random 1 AirportLimo. Try to put as many customers on this limo as possible and update the limo's remaining seats. For simplicity, the thread will call only 1 AirportLimo in each day. Outbound customers who can't get on the limo will be sent to the metro instead.
 - 4.2.3 Reset outbound customers to 0.
- 4.3 Wait until all threads complete outbound processing (if applicable), then process inbound customers:
 - 4.3.1 Random and report inbound arrival (between min & max parameters read from config file). Wait until all threads complete this step.
 - 4.3.2 Random 1 CityLimo. Try to put as many customers on this limo as possible and update the limo's remaining seats. For simplicity, the thread will call only 1 CityLimo in each day. Inbound customers who can't get on the limo will be sent to the metro instead. Wait until all threads complete this step.
 - 4.3.3 Random 1 local guide (i.e. GuideThread) and send all inbound customers to this guide. Update the guide's today customers (accumulated daily) and total customers (accumulated through all days of simulation).
 - 4.3.4 Inbound customers are also added to outbound customers that will be processed on the next even day (step 4.2). But the inbound itself is reset to random arrival every day (step 4.3.1).

**** All TourThreads must work on the same lists of CityLimos, AirportLimos, and GuideThreads.**

5. Implement **class GuideThread** representing an individual local guide as thread. Thread activities are done in loop. Each iteration of a loop = 1 day. In each day:

- 5.1 Wait until one thread (can be main, TourThread, or GuideThread) finishes activities in 4.1.
- 5.2 Also wait until all TourThreads finish sending inbound customers to random GuideThreads. Then report total customers it receives today. A guide may receive customers from >1 TourThreads, or may not receive any customer at all. Wait until all threads complete this step.
- 5.3 In case that it has customers today, random 1 Place and take all the customers to this place. Update the place's total visitors (accumulated through all days of simulation).
- 5.4 Reset today customers to 0.

**** All GuideThreads must work on the same list of Places.**

6. Implement main class with main method.

6.1 Read simulation parameters from config.txt

6.2 Create CityLimos, AirportLimos, Places, TourThreads, GuideThreads according to the parameters. You are recommended to use ArrayLists to keep objects for flexibility. Start all threads.

6.3 After all threads complete all days of simulation, let main thread report local guides' popularity as in the demo. The report must be sorted in decreasing order of total customers, and by thread names (if total customers are equal).

**** Everything printed to the screen must be labelled by the name of the thread who prints it. Don't hard code thread's name but use `Thread.currentThread().getName()`.**

7. Package and folder structure must be correct.

7.1 Your source files (.java) must be in folder Project2_XXX where XXX = full ID of the group representative, assuming that this folder is under Maven's "src/main/java" structure. The first lines of all source files must be comments containing names & IDs of all members.

7.2 Input files must be read from Project2_XXX. Don't use absolute path that is valid only on your PC.

7.3 Add readme.txt containing names & IDs of all members in Project2_XXX.

Submission

1. Group representative zips and submits Project2_XXX to Google classroom

2. Other members submit only readme.txt to Google classroom

Grading

2.5 points daily processing by main + TourThreads (4.1, 4.2.1, 4.2.2, 4.3.2, 4.3.3)

1.5 points daily processing by GuideThreads (5.1, 5.2, 5.3)

2 points summary by main + others (6.3, missing file, thread name)

4 points proper design & programming in OOP style

Maximum & safe concurrency is expected. Don't enforce sequential execution when threads can work in parallel. For example, if 2 TourThreads want to put their customers on 2 different limos, they should be able to do it concurrently.

Late submission: -0.5 points for <1 week late; -1 point for each 1 full week late

```

--- exec:3.1.0:exec (default-cli) @ solutions ---
java.io.FileNotFoundException: src\main\java\Project2\config.txt (The system cannot find the file specified)
New file name =
config
java.io.FileNotFoundException: src\main\java\Project2\config (The system cannot find the file specified)
New file name =
config1
java.io.FileNotFoundException: src\main\java\Project2\config1 (The system cannot find the file specified)
New file name =
config_1.txt
    main >> ===== Parameters =====
    main >> Days of simulation      : 6
    main >> City limo services      : [CityLimo_0, CityLimo_1, CityLimo_2]
    main >> City limo capacity      : 100 per service
    main >> Airport limo services    : [AirportLimo_0, AirportLimo_1]
    main >> Airport limo capacity    : 200 per service
    main >> Places                  : [Place_0, Place_1, Place_2]
    main >> TourThreads             : [TourThread_0, TourThread_1, TourThread_2]
    main >> Daily arrival          : min = 50, max = 100
    main >> GuideThreads           : [GuideThread_0, GuideThread_1, GuideThread_2, GuideThread_3]
    main >>
    main >> =====
    main >> Day 1
    main >> reset CityLimo_0      remaining seats = 100
    main >> reset CityLimo_1      remaining seats = 100
    main >> reset CityLimo_2      remaining seats = 100
    main >> reset AirportLimo_0   remaining seats = 200
    main >> reset AirportLimo_1   remaining seats = 200
    main >>
    TourThread_1 >> inbound customers = 82
    TourThread_2 >> inbound customers = 53
    TourThread_0 >> inbound customers = 54
    TourThread_0 >> put 54 customers on CityLimo_0      remaining seats = 46
    TourThread_1 >> put 82 customers on CityLimo_2      remaining seats = 18
    TourThread_2 >> put 46 customers on CityLimo_0      remaining seats = 0
    TourThread_2 >> put 7 customers on metro
    TourThread_2 >> send 53 customers to GuideThread_2
    TourThread_1 >> send 82 customers to GuideThread_1
    TourThread_0 >> send 54 customers to GuideThread_2
    TourThread_0 >>
    GuideThread_0 >> total customer today = 0
    GuideThread_3 >> total customer today = 0
    GuideThread_1 >> total customer today = 82
    GuideThread_2 >> total customer today = 107 = 53 + 54
    GuideThread_2 >> take 107 customers to Place_1      total visitors = 107
    GuideThread_1 >> take 82 customers to Place_2      total visitors = 82
    main >>
    main >> =====
    main >> Day 2
    main >> reset CityLimo_0      remaining seats = 100
    main >> reset CityLimo_1      remaining seats = 100
    main >> reset CityLimo_2      remaining seats = 100
    main >> reset AirportLimo_0   remaining seats = 200
    main >> reset AirportLimo_1   remaining seats = 200
    main >>
    TourThread_1 >> outbound customers = 82
    TourThread_0 >> outbound customers = 54
    TourThread_2 >> outbound customers = 53
    TourThread_2 >> put 53 customers on AirportLimo_1      remaining seats = 147
    TourThread_1 >> put 82 customers on AirportLimo_0      remaining seats = 118
    TourThread_0 >> put 54 customers on AirportLimo_1      remaining seats = 93
    TourThread_0 >>
    TourThread_0 >> inbound customers = 77
    TourThread_1 >> inbound customers = 76
    TourThread_2 >> inbound customers = 84
    TourThread_2 >> put 84 customers on CityLimo_2      remaining seats = 16
    TourThread_1 >> put 16 customers on CityLimo_2      remaining seats = 0
    TourThread_1 >> put 60 customers on metro
    TourThread_0 >> put 0 customers on CityLimo_2      remaining seats = 0
    TourThread_0 >> put 77 customers on metro
    TourThread_0 >> send 77 customers to GuideThread_3
    TourThread_2 >> send 84 customers to GuideThread_1
    TourThread_1 >> send 76 customers to GuideThread_1
    TourThread_1 >>

```

Missing file handling

Reset every day

Must equal daily inbound

GuideThreads who have customers

= inbound from day 1

```

GuideThread_1 >> total customer today = 160
GuideThread_2 >> total customer today = 0
GuideThread_3 >> total customer today = 77
GuideThread_0 >> total customer today = 0
GuideThread_1 >> take 160 customers to Place_2          total visitors = 242 accumulated from all days
GuideThread_3 >> take 77 customers to Place_0          total visitors = 77
    main >>
    main >> =====
    main >> Day 3
    main >> reset CityLimo_0      remaining seats = 100
    main >> reset CityLimo_1      remaining seats = 100
    main >> reset CityLimo_2      remaining seats = 100
    main >> reset AirportLimo_0 remaining seats = 200
    main >> reset AirportLimo_1 remaining seats = 200
    main >>
TourThread_1 >> inbound customers = 85
TourThread_0 >> inbound customers = 89
TourThread_2 >> inbound customers = 62
TourThread_2 >> put 62 customers on CityLimo_2          remaining seats = 38
TourThread_1 >> put 85 customers on CityLimo_1          remaining seats = 15
TourThread_0 >> put 38 customers on CityLimo_2          remaining seats = 0
TourThread_0 >> put 51 customers on metro
TourThread_0 >> send 89 customers to GuideThread_2
TourThread_1 >> send 85 customers to GuideThread_3
TourThread_2 >> send 62 customers to GuideThread_2
TourThread_2 >>
GuideThread_3 >> total customer today = 85
GuideThread_1 >> total customer today = 0
GuideThread_2 >> total customer today = 151
GuideThread_0 >> total customer today = 0
GuideThread_3 >> take 85 customers to Place_0          total visitors = 162
GuideThread_2 >> take 151 customers to Place_1          total visitors = 258
    main >>
    main >> =====
    main >> Day 4
    main >> reset CityLimo_0      remaining seats = 100
    main >> reset CityLimo_1      remaining seats = 100
    main >> reset CityLimo_2      remaining seats = 100
    main >> reset AirportLimo_0 remaining seats = 200
    main >> reset AirportLimo_1 remaining seats = 200
    main >>
TourThread_2 >> outbound customers = 146 = inbounds from days 2+3 = 84 + 62
TourThread_1 >> outbound customers = 161
TourThread_0 >> outbound customers = 166
TourThread_0 >> put 166 customers on AirportLimo_1      remaining seats = 34
TourThread_1 >> put 34 customers on AirportLimo_1      remaining seats = 0
TourThread_1 >> put 127 customers on metro
TourThread_2 >> put 0 customers on AirportLimo_1      remaining seats = 0
TourThread_2 >> put 146 customers on metro
TourThread_2 >>
TourThread_0 >> inbound customers = 96
TourThread_1 >> inbound customers = 70
TourThread_2 >> inbound customers = 74
TourThread_2 >> put 74 customers on CityLimo_1          remaining seats = 26
TourThread_0 >> put 96 customers on CityLimo_0          remaining seats = 4
TourThread_1 >> put 4 customers on CityLimo_0          remaining seats = 0
TourThread_1 >> put 66 customers on metro
TourThread_1 >> send 70 customers to GuideThread_0
TourThread_2 >> send 74 customers to GuideThread_1
TourThread_0 >> send 96 customers to GuideThread_0
TourThread_0 >>
GuideThread_2 >> total customer today = 0
GuideThread_0 >> total customer today = 166
GuideThread_1 >> total customer today = 74
GuideThread_3 >> total customer today = 0
GuideThread_1 >> take 74 customers to Place_1          total visitors = 332
GuideThread_0 >> take 166 customers to Place_0          total visitors = 328
    main >>

```

```

main >> =====
main >> Day 5
main >> reset CityLimo_0    remaining seats = 100
main >> reset CityLimo_1    remaining seats = 100
main >> reset CityLimo_2    remaining seats = 100
main >> reset AirportLimo_0 remaining seats = 200
main >> reset AirportLimo_1 remaining seats = 200
main >>
TourThread_2 >> inbound customers = 98
TourThread_0 >> inbound customers = 85
TourThread_1 >> inbound customers = 88
TourThread_1 >> put 88 customers on CityLimo_0    remaining seats = 12
TourThread_0 >> put 85 customers on CityLimo_1    remaining seats = 15
TourThread_2 >> put 98 customers on CityLimo_2    remaining seats = 2
TourThread_2 >> send 98 customers to GuideThread_0
TourThread_1 >> send 88 customers to GuideThread_2
TourThread_0 >> send 85 customers to GuideThread_1
TourThread_0 >>
GuideThread_0 >> total customer today = 98
GuideThread_1 >> total customer today = 85
GuideThread_2 >> total customer today = 88
GuideThread_3 >> total customer today = 0
GuideThread_0 >> take 98 customers to Place_2    total visitors = 340
GuideThread_1 >> take 85 customers to Place_0    total visitors = 413
GuideThread_2 >> take 88 customers to Place_2    total visitors = 428
main >>
main >> =====
main >> Day 6
main >> reset CityLimo_0    remaining seats = 100
main >> reset CityLimo_1    remaining seats = 100
main >> reset CityLimo_2    remaining seats = 100
main >> reset AirportLimo_0 remaining seats = 200
main >> reset AirportLimo_1 remaining seats = 200
main >>
TourThread_1 >> outbound customers = 158
TourThread_2 >> outbound customers = 172
TourThread_0 >> outbound customers = 181
TourThread_0 >> put 181 customers on AirportLimo_1    remaining seats = 19
TourThread_2 >> put 172 customers on AirportLimo_0    remaining seats = 28
TourThread_1 >> put 19 customers on AirportLimo_1    remaining seats = 0
TourThread_1 >> put 139 customers on metro
TourThread_1 >>
TourThread_1 >> inbound customers = 81
TourThread_0 >> inbound customers = 83
TourThread_2 >> inbound customers = 74
TourThread_0 >> put 83 customers on CityLimo_2    remaining seats = 17
TourThread_2 >> put 74 customers on CityLimo_1    remaining seats = 26
TourThread_1 >> put 17 customers on CityLimo_2    remaining seats = 0
TourThread_1 >> put 64 customers on metro
TourThread_1 >> send 81 customers to GuideThread_2
TourThread_2 >> send 74 customers to GuideThread_3
TourThread_0 >> send 83 customers to GuideThread_0
TourThread_0 >>
GuideThread_2 >> total customer today = 81
GuideThread_3 >> total customer today = 74
GuideThread_1 >> total customer today = 0
GuideThread_0 >> total customer today = 83
GuideThread_0 >> take 83 customers to Place_0    total visitors = 496
GuideThread_2 >> take 81 customers to Place_2    total visitors = 509
GuideThread_3 >> take 74 customers to Place_2    total visitors = 583
main >>
main >> =====
main >> Summary
main >> GuideThread_2    total customers = 427
main >> GuideThread_1    total customers = 401
main >> GuideThread_0    total customers = 347
main >> GuideThread_3    total customers = 236

```

} Accumulated from all days + sorted

BUILD SUCCESS
